Software Design Document
UART Driver — STM32F411RE

Document ID	SDD-UART_Driver-v1.0
Author	L.A. Lugo Muñiz, 37915 — J.M. Páez Sandoval, 37945
Co-Authors	D. Lopez Moreno, 37984 — C.A. Martínez Macías, 33369
Course	Microcontrollers and Embedded Systems Lab
Institution	CETYS Universidad, Tijuana B.C.
Instructor	Carlos A. Villarreal
Date	May 29, 2026
Status	Released
Target HW	NUCLEO-F411RE (STM32F411RE, Cortex-M4)
Reference SRS	SRS-UART_Driver
Reference RM	RM0383 Rev.3 §19 — USART

---

1. INTRODUCTION

1.1. Purpose

This Software Design Document (SDD) describes the internal design and implementation of the UART Driver module (`uart_driver.h / uart_driver.c`) for the STM32F411RE microcontroller. It captures design decisions, register-level implementation details, and traceability to the UART Software Requirements Specification (SRS-UART_Driver).

1.2. Scope

The UART Driver covers the USART2 peripheral, which is permanently routed to the ST-Link virtual COM port on the NUCLEO-F411RE (PA2 = TX, PA3 = RX). The driver operates in polling mode, exposing a two-function API: `uart_init()` and `uart_write()`. It runs at 16 MHz HSI (no PLL) and targets 115200 baud.

1.3. Definitions

BRR: Baud Rate Register — controls the USART clock divisor

CR1: Control Register 1 — enables TE, RE, UE bits

DR: Data Register — byte written triggers transmission

SR: Status Register — TXE flag indicates DR is empty

TXE: Transmit Data Register Empty (SR bit 7)

UE: USART Enable (CR1 bit 13)

TE: Transmitter Enable (CR1 bit 3)

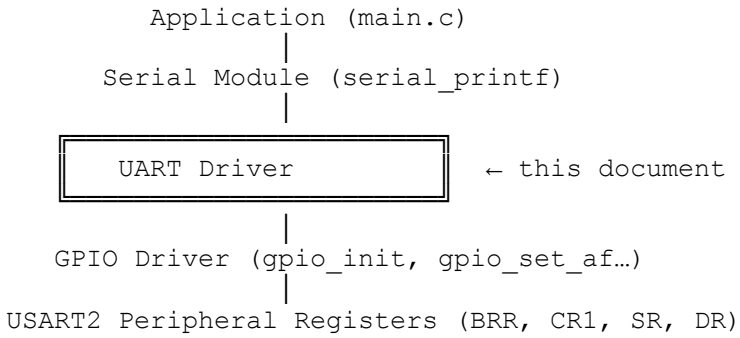
AF7: Alternate Function 7 — routes PA2/PA3 to USART2

PCLK1: APB1 peripheral clock = 16 MHz (HSI, PPRE1 = /1)

2. ARCHITECTURE AND CONTEXT

2.1. Module Position in the Layered Architecture

The UART Driver sits at the hardware abstraction layer, directly above the GPIO Driver. It is consumed by the Serial module, which provides the `serial_printf()` API to the application. No higher-level module accesses USART2 registers directly.



2.2. Hardware Mapping

Signal	MCU Pin	AF	Direction	Notes
USART2_TX	PA2	AF7	Output (Push-Pull)	ST-Link VCP TX
USART2_RX	PA3	AF7	Input (Pull-Up)	ST-Link VCP RX
USART2 clk	APB1	—	—	RCC->APB1ENR bit 17

3. MODULE DECOMPOSITION

3.1. Files

File	Role
uart_driver.h	Public API — types, constants, function prototypes
uart_driver.c	Implementation — register sequences, BRR value

3.2. Types and Constants

```
/* Status type */
typedef enum { UART_OK = 0U, UART_ERR = 1U } uart_status_t;

/* Pre-calculated BRR values (PCLK1 = 16 MHz, OVER8 = 0) */
#define UART_BAUD_115200  (139U)   /* Mantissa=8, Fraction=11 */
#define UART_BAUD_9600    (1667U)  /* Mantissa=104, Fraction=3  */
```

3.3. BRR Calculation

With PCLK1 = 16 MHz and OVER8 = 0 (16× oversampling), the USART divisor is:

```
USARTDIV = 16,000,000 / (16 × 115,200) = 8.680

DIV_Mantissa = 8  →  BRR[15:4] = 0x008

DIV_Fraction = round(0.680 × 16) = 11  →  BRR[3:0] = 0xB

BRR = (8 << 4) | 11 = 0x008B = 139
```

The resulting actual baud rate is 16,000,000 / (16 × 8.6875) = 115,107 baud, an error of 0.08 % — well within the ±2 % tolerance per RM0383 §19.3.4.

4. INTERFACE DESIGN

4.1. Public API

Function	Signature	Returns	FR
uart_init	uart_init(uint32_t brr)	uart_status_t	FR-1, FR-2, FR-3
uart_write	uart_write(uint8_t byte)	void	FR-4, FR-5

4.2. Dependencies

Module Used	Functions Called	Purpose
gpio_driver	gpio_init, gpio_set_output_type, gpio_set_speed, gpio_set_pupd, gpio_set_af	Configure PA2/PA3 as AF7
stm32f411xe.h (CMSIS)	USART2, RCC (register pointers)	Direct peripheral register access

5. DETAILED DESIGN

5.1. uart_init()

uart_init

Signature: `uart_status_t uart_init(uint32_t brr)`

Purpose: Configures GPIO pins, enables USART2 clock, programs BRR, and enables the USART transmitter and receiver.

Implementation steps:

1. Enable GPIOA clock: RCC->AHB1ENR |= GPIOAEN (via gpio_init).
2. Configure PA2 as AF7, push-pull, high speed, no pull (TX).
3. Configure PA3 as AF7, push-pull, high speed, pull-up (RX idle = HIGH).
4. Enable USART2 APB1 clock: RCC->APB1ENR |= RCC_APB1ENR_USART2EN.
5. Write BRR register with the pre-calculated divisor value.
6. Write CR1: set TE (bit 3) and RE (bit 2) first, then UE (bit 13) last — RM0383 §19.3.2 requires UE after TE/RE.
7. Return UART_OK.

Register operations:

Register	Bit Field	Value	Effect
`RCC->AHB1ENR`	`bit 0 (GPIOAEN)`	`|= 1`	Enable GPIOA clock
`GPIOA->MODER`	`bits 5:4 (PA2)`	`= 10b`	PA2 → Alternate Function mode
`GPIOA->MODER`	`bits 7:6 (PA3)`	`= 10b`	PA3 → Alternate Function mode
`GPIOA->AFR[0]`	`bits 11:8 (PA2)`	`= 0x7`	PA2 → AF7 (USART2_TX)
`GPIOA->AFR[0]`	`bits 15:12 (PA3)`	`= 0x7`	PA3 → AF7 (USART2_RX)
`RCC->APB1ENR`	`bit 17 (USART2EN)`	`|= 1`	Enable USART2 APB1 clock
`USART2->BRR`	`bits 15:0`	`= 139`	Set baud rate to 115200
`USART2->CR1`	`bit 3 (TE)`	`|= 1`	Enable transmitter
`USART2->CR1`	`bit 2 (RE)`	`|= 1`	Enable receiver
`USART2->CR1`	`bit 13 (UE)`	`|= 1`	Enable USART peripheral

5.2. uart_write()

uart_write

Signature: `void uart_write(uint8_t byte)`

Purpose: Transmits one byte over USART2 using polling. Blocks until the transmit data register is empty, then writes the byte to DR.

Implementation steps:

1. Poll SR.TXE (bit 7) in a spin-loop: while (!(USART2->SR & USART_SR_TXE)).
2. When TXE = 1, DR is empty and ready to accept new data.
3. Write the byte: USART2->DR = (uint32_t)(byte & 0xFF).
4. Writing DR clears TXE by hardware; the shift register begins transmitting immediately.
5. Return (TC not polled — caller serializes characters via TXE polling on the next call).

Register operations:

Register	Bit Field	Value	Effect
`USART2->SR`	`bit 7 (TXE)`	`read`	Wait until DR is empty
`USART2->DR`	`bits 7:0`	`= byte`	Load character into shift register

6. FUNCTIONAL REQUIREMENTS COMPLIANCE

FR	Requirement	Function / Register	Status
FR-1	Enable USART2 peripheral clock (APB1)	RCC->APB1ENR |= USART2EN	✓ Satisfied
FR-2	Configure baud rate via BRR register	USART2->BRR = 139 (115200 baud)	✓ Satisfied
FR-3	Enable TX (TE) and peripheral (UE) in CR1	CR1 = TE | RE | UE	✓ Satisfied
FR-4	Poll SR.TXE before writing DR	while(!(SR & TXE));	✓ Satisfied
FR-5	Write 8-bit data to DR	DR = byte & 0xFF	✓ Satisfied

7. DESIGN DECISIONS

Decision	Rationale
Polling (not DMA/IRQ) for uart_write	Simplest correct implementation for Lab 4 scope. Polling is safe for the 500 ms telemetry period; CPU blocking time per character is < 87 µs at 115200 baud.
BRR as a #define constant, not computed at runtime	PCLK1 is fixed at 16 MHz HSI for this project. A compile-time constant eliminates division at runtime and is transparent to future readers.
UE set after TE and RE	RM0383 §19.3.2 states UE must be the last bit set in CR1. Writing all three in a single OR operation may cause transient states; sequential writes are safer.
gpio_set_pupd(PA3, PULLUP)	UART idle line is HIGH. Without pull-up, PA3 floats during reset and before the ST-Link establishes the VCP, causing false start bits.
RE enabled even though RX is not used in Lab 4	Enables bidirectional future use without re-initialization, and matches the NUCLEO default VCP configuration.

8. REFERENCES

[1] STMicroelectronics, "STM32F411xC/E Reference Manual (RM0383)," Rev. 3, 2019 -- Sec. 19 USART.

[2] STMicroelectronics, "NUCLEO-F411RE User Manual (UM1724)," Rev. 14, 2021.

[3] SRS-UART_Driver, MasterCarlosV/MSE_LabCourse, GitHub.